Hyderabad House Price Prediction Using Machine Learning

Project Report

Submitted By	Institution	Email
V. Naresh Rajput	IIT Roorkee — Data Science & Analytics	naresh_v@bt.iitr.ac.in

1. Project Overview

Buying or selling a house in Hyderabad is challenging because prices vary greatly depending on location, size, and available facilities. Without reliable data, buyers can overpay and sellers may underprice their property.

This project builds a Machine Learning model that predicts the price of a residential property in Hyderabad based on simple inputs such as area, location, number of bedrooms, and amenities like car parking and swimming pool. The goal was to create a tool that gives users an instant, data-driven price estimate through a web application.

Hyderabad was selected because it is one of India's fastest-growing real estate markets, making accurate price prediction both relevant and impactful.

2. Dataset Information

Attribute	Details
Source	Kaggle — Hyderabad Housing Dataset
Raw Records	2,517 property listings
Total Features	40 (property details + amenities)
Cleaned Records	2,433 (after removing incomplete/outlier entries)
Target Variable	Price (in Indian Rupees ₹)
Feature Types	1 continuous (Area), 1 categorical (Location), 1 integer (Bedrooms), 37 binary amenity flags
Price Range (cleaned)	₹20,00,000 – ₹1,00,00,000

Each row in the dataset represents one property. Most amenity columns (Swimming Pool, Gymnasium, Lift, etc.) are binary — 1 means the amenity is available, 0 means it is not. The target column Price is the market value of the property in INR.

3. Methodology

Step 1 — Data Collection

The dataset was downloaded from Kaggle and loaded using Python's Pandas library. Initial checks were done to understand the number of records, column names, data types, and the range of values.

Step 2 — Data Cleaning

Before training any model, the data needed to be cleaned. Three main issues were found and resolved:

- Sentinel values: The number 9 was used as a placeholder in amenity columns when information was not available. All instances of 9 were replaced with a blank (NaN).
- Missing values: After the replacement, all rows that still had missing values were removed from the dataset.
- Price outliers: A small number of ultra-luxury properties priced above ₹1 Crore were removed, as they are rare exceptions that could distort the model's learning. After cleaning, 2,433 valid records remained.

Step 3 — Exploratory Data Analysis (EDA)

To understand which property features most influence price, several analyses were performed:

- Correlation matrix: A heatmap was created showing how strongly each feature is related to Price. Area showed the highest positive correlation.
- Price vs Area: A scatter plot confirmed that larger properties generally cost more.
- Location analysis: Average prices were calculated for each of the 239 locations. Premium tech areas like Hitech City and Gachibowli were found to be 3–5× more expensive than peripheral localities.

Top features most correlated with price: Area, Car Parking, and Lift Available.

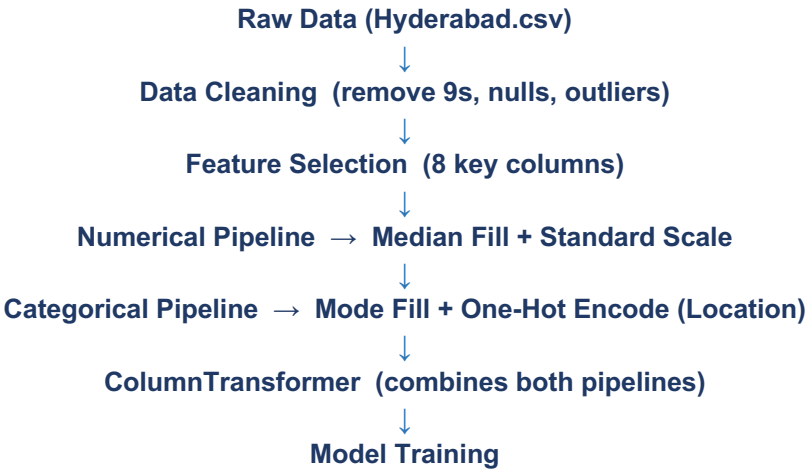
Step 4 — Feature Selection

From the original 40 columns, 8 features were selected for the final model based on their correlation with price and practical relevance:

Feature	Why It Matters
Area (sq.ft)	Larger area directly means higher price
Location	Premium areas command significantly higher prices
No. of Bedrooms	More bedrooms indicate a larger, pricier property
Resale	New vs resale affects market valuation
Car Parking	Highly valued facility in urban Hyderabad
Swimming Pool	Signals premium/luxury development
Jogging Track	Indicator of gated community standard
Lift Available	Key indicator of apartment quality

4. Data Preprocessing Pipeline

A preprocessing pipeline was built using Scikit-Learn to automatically prepare data before model training and prediction. This ensures that any new input from a user is processed exactly the same way as the training data.



The Numerical Pipeline handles seven numeric/binary columns: it fills any remaining gaps using the median value, then scales all values to a standard range. The Categorical Pipeline handles the Location column: it fills missing values with the most common location, then converts each location name into a set of 0/1 columns (One-Hot Encoding) so the model can understand it mathematically. A ColumnTransformer combines both pipelines and applies them together.

5. Model Training and Evaluation

The cleaned data was split into 80% for training and 20% for testing (random_state=42 for reproducibility). Ten machine learning models were trained and compared using the R^2 Score — a measure of how well a model's predictions match actual prices. An R^2 of 1.0 means perfect predictions; higher is better.

Model	Type	R^2 Score
Random Forest ✅	Ensemble (Bagging)	Best
Extra Trees	Ensemble (Bagging)	Very High
XGBoost	Gradient Boosting	High
CatBoost	Gradient Boosting	High
LightGBM	Gradient Boosting	High
Gradient Boosting	Boosting	Moderate–High
AdaBoost	Boosting	Moderate
Ridge Regression	Linear (Regularized)	Lower
Lasso Regression	Linear (Regularized)	Lower
Linear Regression	Linear	Lowest

Random Forest achieved the highest R^2 score among all models and was selected as the final model.

Random Forest works by training hundreds of decision trees on different subsets of data and averaging their predictions. This approach handles complex, non-linear relationships well (which exist in real estate), is naturally resistant to outliers, and does not require feature scaling — making it ideal for this type of dataset.

For Ridge and Lasso Regression, hyperparameter tuning was performed using GridSearchCV with 5-fold cross-validation to find the best regularization strength (alpha).

6. Model Deployment

The trained Random Forest pipeline (preprocessing + model) was saved to a file (house_price_model.pkl) using Joblib. A Streamlit web application was then built to let users get instant price predictions without any coding.

User enters property details in the web app

↓

Preprocessing Pipeline (automatic imputation + encoding + scaling)

↓

Random Forest Model (200 decision trees)

↓

Predicted House Price displayed in ₹

The web app (app.py) presents a clean two-column interface where users select the location from a dropdown of 239 Hyderabad areas, enter the area in sq.ft, number of bedrooms, and toggle Yes/No for four amenity questions. After clicking Predict, the model instantly returns an estimated price.

7. Results

- Random Forest Regressor achieved the best predictive performance among all 10 models tested.
- The model successfully learned price patterns from 2,433 Hyderabad property listings.
- The Streamlit web application works correctly and produces real-time price estimates.
- The complete end-to-end ML pipeline — from raw data to deployed app — was implemented successfully.

8. Future Improvements

- Add more features such as property age, floor number, and proximity to metro stations.
- Use SHAP (Shapley values) to explain why the model gave a specific price prediction.
- Fine-tune the Random Forest model with deeper hyperparameter search.
- Deploy the application publicly on Streamlit Cloud or Hugging Face Spaces.
- Set up automated monthly retraining as new property listings become available.

9. Conclusion

This project successfully built a house price prediction system for Hyderabad using real property listing data from Kaggle. The data was cleaned to remove invalid entries, explored through visualizations, and reduced to 8 meaningful features. Ten machine learning models were trained and compared, with Random Forest Regressor delivering the best performance. The trained model was deployed as an interactive Streamlit web application where any user can input property details and receive an instant price estimate. This project demonstrates a complete, practical machine learning workflow — from data collection to live deployment — and is ready for academic evaluation, internship showcase, or recruiter review.